

SÉCURITÉ DES SYSTÈMES

M1 CSIA (S2)

M. MEDEDJEL

2025-2026

COURS 6 (CHAPITRE IV)

LES PARADES ET LES TECHNIQUES D'ÉVASION

Explication

- ❑ **Parades** : mécanismes de défense et contre-mesures (pare-feu, IDS/IPS, durcissement, etc.).
- ❑ **Techniques d'évasion** : méthodes d'attaque visant à contourner ou déjouer ces protections.

➔ A retenir:

« **Parades** » = ce que fait le **défenseur**.

« **Techniques d'évasion** » = ce que fait l'**attaquant** pour passer malgré les parades.

OBJECTIFS

Les objectifs de ce chapitre peuvent être résumés dans ce qui suit :

- ❑ Définir de quoi il s'agit le concept de la défense en profondeur (*defense in depth*)
- ❑ Décrire les différentes couches de parades et leurs rôles.
- ❑ Comprendre le fonctionnement de quelques types de parades et les techniques d'évasion utilisées.

INTRODUCTION

Les systèmes informatiques sont confrontés à de nombreuses menaces et vulnérabilités.

...

Comment peut-on construire des systèmes réellement sécurisés ?

INTRODUCTION

Dans ce cours, nous verrons comment renforcer la sécurité des systèmes.

- Notion clé : **défense en profondeur (defense in depth)**.
- Principe : déployer **plusieurs couches de protection** successives.

Ainsi, même si une barrière est franchie, l'attaquant se heurte à d'autres obstacles, ce qui limite le risque de compromission totale.

LES PARE-FEUX

- Les **pare-feux (firewalls)** protègent un système ou un ensemble de systèmes contre les menaces venant du réseau.
- Ils contrôlent le trafic entrant et sortant selon des règles de sécurité définies.
- Ils permettent de limiter les attaques externes tout en autorisant un accès contrôlé aux réseaux étendus et à Internet.

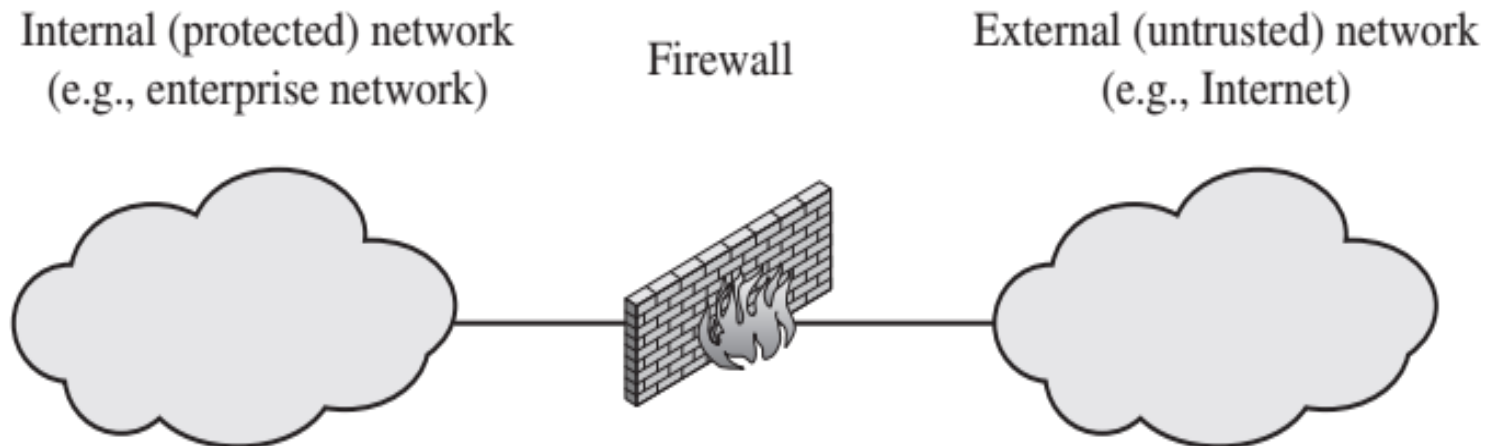


Fig. 1. Modèle général d'un Pare-feu

LES PARE-FEUX

Pourquoi les pare-feux ?

- Les pare-feux sont nécessaires dans des environnements de plus en plus **ouverts et interconnectés** :
 - **Systèmes centraux** de traitement de données exposés au réseau.
 - **Réseaux locaux (LAN)** répartis géographiquement et reliés via des **réseaux étendus (WAN)**.
 - **Informatique en nuage (cloud computing)** : serveurs virtualisés dans des centres de données accessibles par **Internet**.
- Dans ces contextes, les pare-feux servent de **barrière de protection** entre les ressources internes et le monde extérieur.

LES PARE-FEUX

Pourquoi les pare-feux ?

Exemple: le pare-feu comme complément à la sécurité hôte

- Dans un grand réseau (centaines/milliers de postes Windows, macOS, Linux), une nouvelle **faille de sécurité** impose de **mettre à jour chaque machine** potentiellement vulnérable.
- Une sécurité uniquement **basée sur l'hôte** (patch sur chaque système) est **indispensable**, mais **lourde à gérer**.
- Les **pare-feux** offrent une **protection supplémentaire centralisée** :
 - ils complètent les mécanismes de sécurité locaux,
 - et réduisent l'exposition globale du système aux attaques réseau.

LES PARE-FEUX

Pourquoi les pare-feux ?

Le pare-feu fournit donc une couche supplémentaire de défense, isolant les systèmes internes des réseaux externes.

Cela correspond à la doctrine militaire classique de la "défense en profondeur", qui s'applique également à la sécurité informatique.

LES PARE-FEUX

Les objectifs de conception d'un pare-feu :

- **Point de passage unique**
 - Tout le trafic entre le **réseau interne** et l'**extérieur** doit passer par le **pare-feu**.
 - Tout accès direct au réseau local en dehors du pare-feu doit être **bloqué**.
- **Filtrage selon la politique de sécurité**
 - Seul le **trafic explicitement autorisé** par la **politique de sécurité** peut passer.
 - Différents types de pare-feux appliquent **différentes politiques** (filtrage IP/ports, inspection applicative, etc.).

LES PARE-FEUX

Les objectifs de conception d'un pare-feu :

- **Pare-feu renforcé**
 - Le pare-feu lui-même doit être **difficile à compromettre**.
 - Il repose sur un **système renforcé** (durcissement) et un **système d'exploitation sécurisé**.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

Un élément essentiel de la mise en œuvre d'un pare-feu consiste à spécifier une **politique d'accès** appropriée.

➔ répertorier les types de trafic autorisés à traverser le pare-feu, y compris les plages d'adresses, les protocoles, les applications et les types de contenu.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

❑ Critères de filtrage selon le NIST SP 800-41

NIST SP 800-41 (*Guidelines on Firewalls and Firewall Policy, 2009*) répertorie une série de caractéristiques qu'une politique d'accès au pare-feu pourrait utiliser pour filtrer le trafic, notamment :

- Adresse IP et valeurs de protocole
- Protocole d'application
- Identité de l'utilisateur
- Activité réseau

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

➤ Adresse IP et valeurs de protocole

- Contrôle l'accès en fonction des adresses (source ou destination), des numéros de port, de la direction du flux (entrant ou sortant) et d'autres caractéristiques réseau (protocole, interface, etc.).
- Ce type de filtrage est utilisé par **les pare-feux à filtrage de paquets** (avec ou sans état) pour limiter l'accès à certains services et ports spécifiques.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

➤ Protocole d'application

- Contrôle l'accès en fonction des données des protocoles d'application.
- Ce type de filtrage est mis en œuvre par des **pare-feux de niveau application / proxies applicatifs**, qui relaient et surveille l'échange d'informations pour des protocoles précis.
- Par exemple:
 - Analyse des **e-mails SMTP** pour détecter les **spam** ou des pièces jointes malveillantes.
 - Filtrage des **requêtes HTTP** pour n'autoriser que certains **sites ou URLs**.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

➤ Identité de l'utilisateur

- Contrôle l'accès en fonction de l'**identité de l'utilisateur** plutôt que seulement de l'adresse IP ou du port.
- Généralement pour les utilisateurs internes qui s'identifient via des mécanismes **sécurisés**, comme IPSec (*Internet Protocol Security*).

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

➤ **Activité réseau**

- Contrôle l'accès en fonction de l'activité réseau observée et de son contexte.
- Exemples de critères:
 - **Heure de la requête** : autoriser certains accès uniquement pendant les heures ouvrables.
 - **Taux de requêtes** : détecter un nombre anormalement élevé de connexions (scan, brute force,...).

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

❑ Fonctionnalités d'un pare-feu

- **Point de contrôle central** : bloque les accès non autorisés et limite les services à risque.
- **Gestion simplifiée** : un point d'étranglement unique pour administrer la sécurité.
- **Surveillance** : journaux, audits, alertes sur le trafic réseau.
- **Fonctions annexes** : **NAT** et support de **VPN/IPSec** (mode tunnel).

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

Limites et défis des pare-feux

1. **Complexité de configuration** : Une mauvaise configuration, surtout dans des environnements complexes, peut **créer des vulnérabilités** ou **dégrader** les performances du réseau.
2. **Limite de visibilité** : Les pare-feux ne peuvent pas toujours détecter les **attaques zero-day** ou les **menaces internes** provenant d'utilisateurs autorisés. Leur fonction principale est de surveiller le trafic réseau externe plutôt que sur les activités internes des utilisateurs.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

Limites et défis des pare-feux

3. **Coût** : Les pare-feux sophistiqués (fonctions avancées, haute performance) peuvent être coûteux à déployer et à maintenir, surtout dans les grandes organisations.
4. **Complexité des politiques** : La gestion des politiques de pare-feu devient complexe à mesure que les règles s'accumulent, ce qui peut entraîner des erreurs de configuration ou des conflits entre les règles.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

Limites et défis des pare-feux

5. **Contournement possible:** Les pare-feux peuvent être contournés par des applications ou protocoles alternatifs, comme les **tunnels chiffrés**, permettant aux utilisateurs d'accéder à des ressources non sécurisées malgré les restrictions.
6. **Limitations de performance :** Les pare-feux peuvent dégrader les performances du réseau, notamment lors d'analyses approfondies du trafic ou en cas de surcharge due à un volume élevé de données.

PARE-FEU : CARACTÉRISTIQUES ET POLITIQUE D'ACCÈS

Limites et défis des pare-feux

7. **Problèmes d'évolutivité** : Dans des réseaux très vastes ou très dynamiques, un pare-feu unique peut atteindre ses **limites**. Il faut alors des **architectures adaptées** (répartition de charge, pare-feux distribués, solutions cloud, etc.) pour passer à l'**échelle**.

TYPES DE PARE-FEU

- Un pare-feu peut surveiller le trafic réseau à plusieurs niveaux, des paquets réseau de bas niveau jusqu'à l'inspection des détails des protocoles d'application.
- Différentes technologies selon :
 - la **couche** du modèle réseau (IP, transport, application),
 - le **mode de fonctionnement** (filtrage simple, avec état, proxy, NGFW, etc.),
 - le **support** (matériel, logiciel, cloud).

TYPES DE PARE-FEU

Logique de filtrage

Le **comportement** du pare-feu est défini par la **politique d'accès** :

- **Filtre positif**, n'autorise que les paquets correspondant à des critères précis (tout le reste est bloqué).
- **Filtre négatif**, bloque les paquets correspondant à certains critères, et laisse passer le reste.

TYPES DE PARE-FEU

Logique de filtrage

- Selon son **type**, le pare-feu peut analyser :
 - uniquement les **en-têtes** des paquets (adresses, ports, protocoles),
 - la **charge utile** (contenu) des paquets,
 - ou encore les **modèles de trafic** produits par une séquence de paquets (comportement global).
- Dans ce qui suit, nous allons examiner les principaux types de pare-feu en fonction de leur **niveau d'analyse** et de leur **mode de filtrage**.

TYPES DE PARE-FEU

➤ Pare-feu à filtrage de paquets

Ce pare-feu applique un ensemble de règles à chaque paquet IP entrant et sortant, puis le transmet ou le rejette.

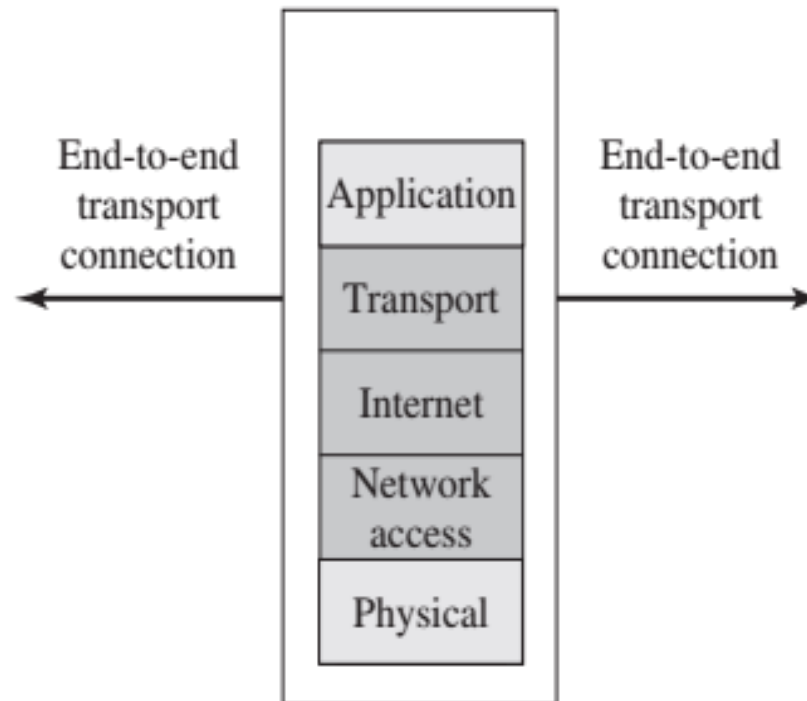


Fig. 2.a. Pare-feu de filtrage de paquets [3]

TYPES DE PARE-FEU

➤ Pare-feu à filtrage de paquets

Les règles de filtrage sont basées sur les informations contenues dans un paquet :

- réseau (@IP src/dest),
- @ src/dest au niveau couche Transport (TCP/UDP port),
- Interface (pare-feu avec trois ports ou plus),
- ...

TYPES DE PARE-FEU

➤ Pare-feu à filtrage de paquets

- Le filtrage de paquets est configuré sous forme d'une **liste de règles** basées sur les champs des en-têtes **IP / TCP / UDP**.
- Pour chaque paquet :
 - s'il **correspond** à une règle, l'action associée est appliquée (**autoriser** ou **bloquer**),
 - s'il ne correspond à **aucune règle**, une **action par défaut** est appliquée.
- Deux stratégies par défaut sont possibles : **rejeter** ou **transmettre**.

TYPES DE PARE-FEU

➤ Pare-feu à filtrage de paquets - Exemple

Le tableau suivant est un exemple simplifié d'un ensemble de règles pour le trafic SMTP. L'objectif est d'autoriser le trafic de courrier entrant et sortant, mais de bloquer tout autre trafic.

Tableau 1 : Exemple de filtrage de paquets

Rule	Direction	Src address	Dest addresss	Protocol	Dest port	Action
1	In	External	Internal	TCP	25	Permit
2	Out	Internal	External	TCP	>1023	Permit
3	Out	Internal	External	TCP	25	Permit
4	In	External	Internal	TCP	>1023	Permit
5	Either	Any	Any	Any	Any	Deny

Question : *Y a-t-il un problème avec cet ensemble de règles ?*

TYPES DE PARE-FEU

➤ Pare-feu à filtrage de paquets - Exemple

- Le jeu de règles actuel présente plusieurs problèmes. En particulier, la **règle 4** autorise le trafic externe vers tout port de destination **> 1023**.
- Exemple d'exploitation :
 - un attaquant externe ouvre une connexion depuis son **port 5150** vers un **proxy Web interne** sur le **port 8080**,
 - ce type d'accès devrait être **interdit**, mais est autorisé par la règle 4, ce qui peut permettre une **attaque sur le serveur interne**.
- ✓ **Contre-mesure :**
 - ajouter un **critère sur le port source** dans chaque règle,
 - par exemple :
 - pour les règles **2 et 4** : **port source = 25**,
 - pour les règles **1 et 3** : **port source > 1023**.

TYPES DE PARE-FEU

➤ Pare-feu à mémoire (*Stateful Firewall*)

- Un pare-feu à filtrage simple (**stateless**) prend sa décision **paquet par paquet**, sans tenir compte de ce qui s'est passé avant ou après.
- Il ne voit pas la **connexion** (session), seulement des paquets isolés.
- Un pare-feu à mémoire (**stateful**), au contraire :
 - **garde en mémoire l'état des connexions** (table d'état),
 - sait si un paquet appartient à une **connexion déjà établie** ou à une tentative **nouvelle / suspecte**,
 - utilise ce **contexte de couche supérieure** (état TCP, sens de la connexion, etc.) pour prendre des décisions de filtrage plus **finas et sûres**.

TYPES DE PARE-FEU

➤ Pare-feu à mémoire (*Stateful Firewall*) – Exemple

Par exemple, avec le protocole SMTP:

- Le **client** envoie les mails, le **serveur** les reçoit et les stocke.
- La connexion TCP se fait typiquement :
 - vers le **port 25** du serveur (port standard SMTP),
 - depuis un **port client dynamique** compris entre **1024 et 65535**.

Règle générale pour TCP

En pratique, beaucoup d'applications TCP utilisent :

- un **port serveur “bien connu”** (< 1024),
- un **port client éphémère** (> 1023).

TYPES DE PARE-FEU

➤ Pare-feu à mémoire (*Stateful Firewall*) – Exemple

- En général, lorsqu'une application utilisant TCP établit une session avec un hôte distant, elle utilise un port serveur standard (**numéro < 1024**) et un port client dynamique (**entre 1024 et 65535**).
- Un simple pare-feu de type filtrage de paquets doit donc autoriser le trafic entrant sur ces ports élevés, ce qui introduit une **vulnérabilité potentielle** pouvant être exploitée par des utilisateurs non autorisés.
- Donc, un pare-feu à mémoire ou à état renforce les règles du trafic TCP en créant un répertoire de connexions TCP, comme indiqué dans le Tableau 2.

TYPES DE PARE-FEU

➤ Pare-feu à mémoire (*Stateful Firewall*)

Tableau 2 : Exemple de tableau d'état de connexion du pare-feu à mémoire

Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.9.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established
219.22.123.32	2112	192.168.1.6	80	Established
210.99.212.18	3321	192.168.1.6	80	Established
24.102.32.23	1025	192.168.1.6	80	Established
223.21.22.12	1046	192.168.1.6	80	Established

Le filtre de paquets autorisera désormais le trafic entrant uniquement pour les paquets qui correspondent au profil de l'une des entrées de ce tableau.

TYPES DE PARE-FEU

➤ Pare-feu à mémoire (*Stateful Firewall*)

Ce pare-feu examine les mêmes informations sur les paquets qu'un pare-feu à filtrage de paquets, mais enregistre également des informations sur les connexions TCP.

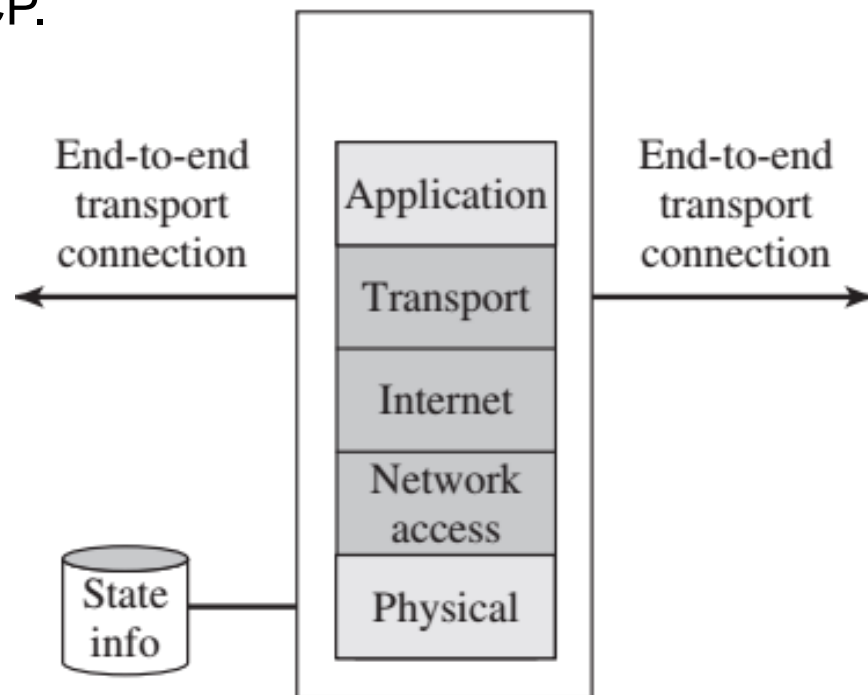


Fig. 2.b. Pare-feu de filtrage de paquets à état [3]

TYPES DE PARE-FEU

➤ Pare-feu d'application

Ce pare-feu, également appelée proxy, agit comme un relais du trafic au niveau de l'application (Figure 2.c).

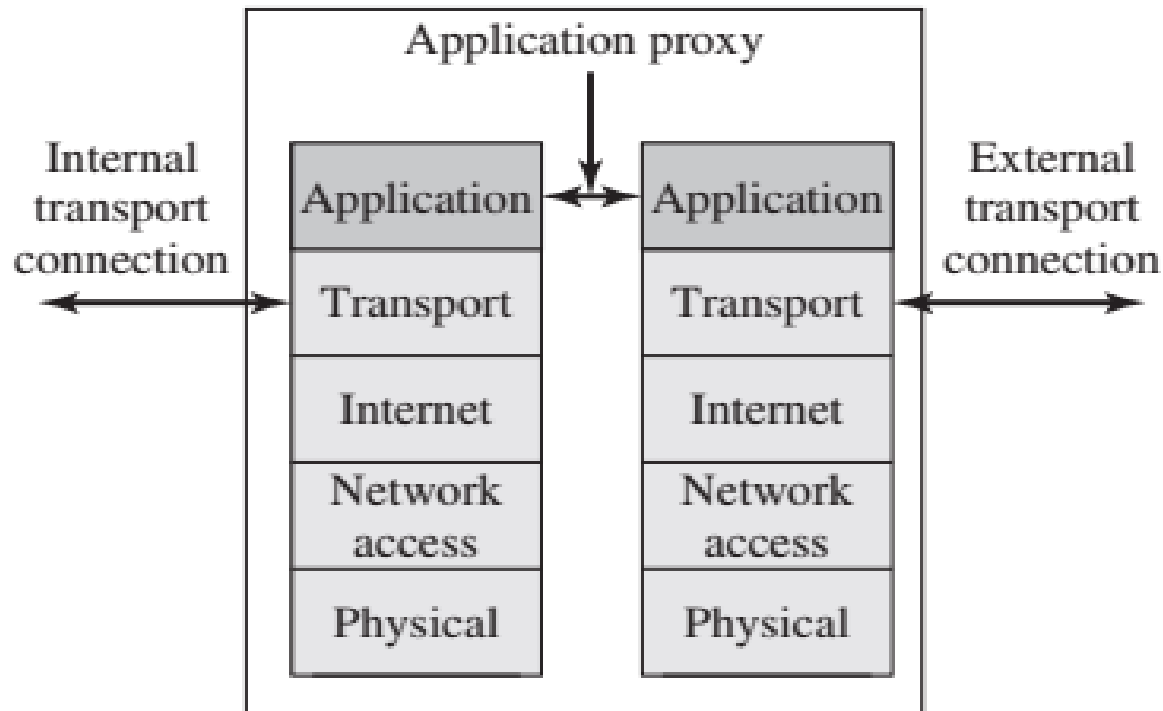


Fig. 2.c. Pare-feu proxy [3]

TYPES DE PARE-FEU

➤ Pare-feu d'application

- L'utilisateur se connecte d'abord à la **passerelle (pare-feu applicatif)** via une application TCP/IP (Telnet, FTP, etc.).
- La passerelle lui demande le **nom de l'hôte distant** puis vérifie son **identité** et ses **informations d'authentification**.
- Une fois l'utilisateur **autorisé**, la passerelle ouvre elle-même la connexion vers l'application sur l'hôte distant et relaye les segments TCP entre le client et le serveur.
- Si aucun **proxy dédié** n'existe pour un service donné, ce service **ne peut pas traverser** le pare-feu : il n'est tout simplement pas autorisé.

TYPES DE PARE-FEU

➤ Pare-feu d'application

- Les **passerelles de niveau application** sont généralement **plus sûres** que les simples filtres de paquets.
- Elles se concentrent sur des **protocoles applicatifs précis** plutôt que seulement sur les **adresses IP** et **ports**.
- Exemple FTP :
 - c'est le **protocole FTP** qui est analysé, pas seulement le **port 21**.
 - la passerelle vérifie que les **commandes et réponses** respectent bien les **spécifications du protocole**.
- Résultat :
 - les paquets **malformés** ou **malveillants** sont bloqués,
 - le filtrage est **plus fin et plus efficace**, ce qui renforce la **sécurité globale**.

TYPES DE PARE-FEU

➤ Pare-feu d'application

Inconvénient principal : la charge de traitement

- La passerelle doit **analyser en profondeur** et **relayer** tout le trafic dans les deux sens.
- Cela consomme beaucoup de **ressources CPU / mémoire**.
- En cas de **trafic important**, cela peut provoquer une **baisse de performance** et devenir un **goulot d'étranglement**.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

Conception du déploiement des pare-feux

- A suivre ...

PARE-FEU : EMPLACEMENT ET CONFIGURATION

Conception du déploiement des pare-feux

- Comme indiqué précédemment (voir figure 1), un pare-feu sert de barrière entre une source de trafic externe potentiellement non fiable et un réseau interne.
- À partir de ce principe, l'administrateur sécurité doit décider :
 - **où placer** les pare-feux,
 - et **combien** de pare-feux déployer.
- Dans cette section, nous présenterons les **configurations courantes** utilisées pour **renforcer la protection du réseau** (pare-feu en bordure, DMZ, pare-feux internes, etc.).

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ (Demilitarized Zone)

➤ Principe général

- Une **DMZ** est une **zone réseau intermédiaire** entre Internet et le réseau interne.
- Elle est protégée par :
 - un **pare-feu externe** (Internet ↔ DMZ),
 - un ou plusieurs **pare-feux internes** (DMZ ↔ réseau local).

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ (Demilitarized Zone)

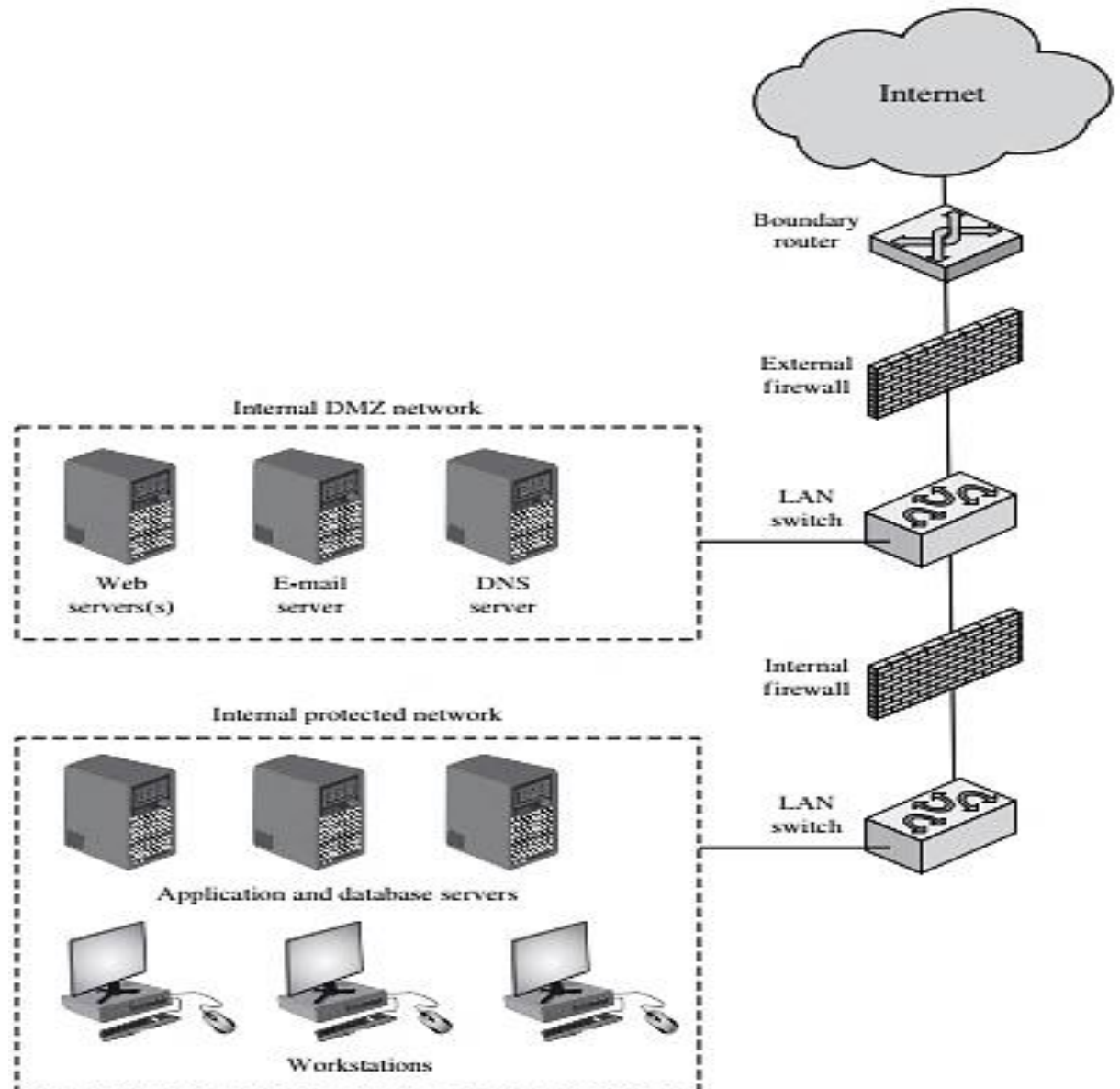
➤ Rôle de la DMZ

- Héberger les services accessibles depuis l'extérieur :
 - **Serveur Web, messagerie, DNS**, etc.
- Ces systèmes doivent :
 - rester ouverts aux connexions externes,
 - être isolés du reste du LAN pour limiter l'impact en cas de compromission.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ

Fig. 3. Exemple de configuration de pare-feu [3]



PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ

➤ Rôle du pare-feu externe

▪ Filtrage Internet

- Contrôle les accès entre Internet et la DMZ.
- Protège les serveurs de la DMZ en fonction de leurs besoins réels de connectivité externe (ports/protocoles strictement nécessaires).

▪ Protection minimale du LAN

- Fournit aussi un premier niveau de protection pour le reste du réseau de l'entreprise (LAN), même si ce n'est pas sa fonction principale.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ

➤ Rôle du pare-feu interne

Son objectif global est de renforcer la séparation entre **DMZ** et **LAN**.

▪ Filtrage plus strict vers le LAN

- Règles plus restrictives que le pare-feu externe.
- Protège les serveurs et postes internes contre les attaques venant d'Internet et transitant par la DMZ.

▪ Protection bidirectionnelle entre DMZ et réseau interne

- Empêche qu'une machine DMZ compromise (vers, rootkits, bots, ...) attaque le réseau interne.
- Empêche des attaques ou accès non autorisés depuis le réseau interne vers la DMZ.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux DMZ

➤ Rôle du pare-feu interne

▪ Segmentation interne avancée

- On peut déployer plusieurs pare-feu internes pour isoler différentes zones du réseau interne (Serveurs, Bureaux, R&D, ...).
- Limite la propagation d'une compromission d'un segment à l'autre.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux privés virtuels (VPN)

▪ Principe général :

- Un **VPN** permet à des **ordinateurs distants** de se connecter à un réseau via un **support peu sécurisé** (ex. Internet).
- Il garantit la **confidentialité** et l'**intégrité** des données échangées.
- Le protocole le plus utilisé dans ce contexte est **IPSec**, qui assure une communication sécurisée au niveau IP (chiffrement, intégrité, authentification).

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux privés virtuels (VPN)

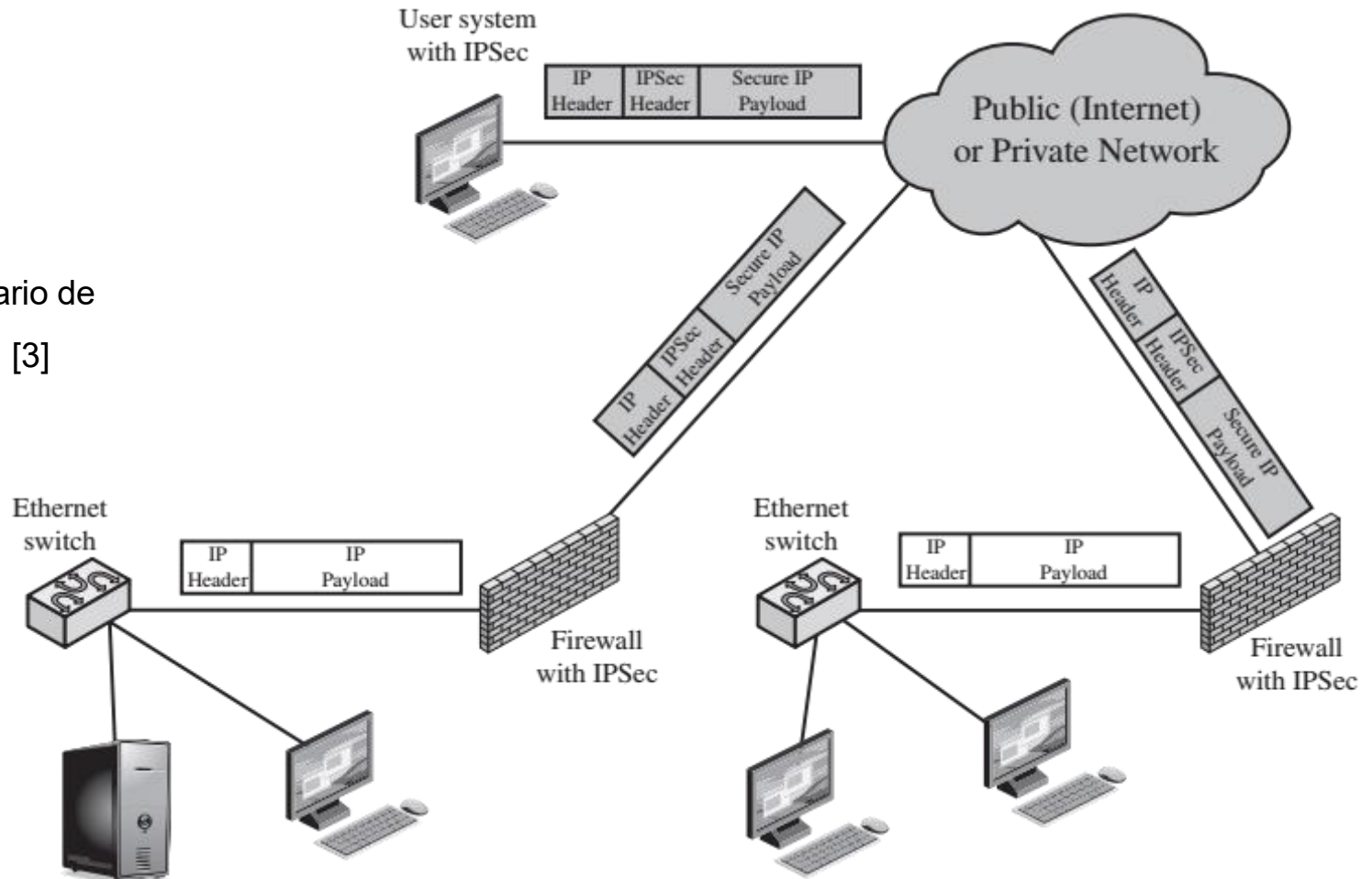


Fig. 4. Un scénario de sécurité VPN [3]

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux privés virtuels (VPN)

La figure 4 illustre un scénario typique d'utilisation d'IPSec:

- **Sur chaque LAN** : Trafic IP non chiffré
- Pour le trafic **hors site (via WAN privé ou public)**: Utilisation d'**IPSec** dans des périphériques réseau (routeurs, pare-feu) reliant les LAN au WAN.

Points clés:

- ✓ Chiffrement et compression du trafic entrant dans le WAN
- ✓ Déchiffrement et décompression du trafic provenant du WAN
- ✓ Authentification possible des échanges
- ✓ Opérations transparentes pour les postes de travail et serveurs.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux privés virtuels (VPN) – Positionnement optimal

- **VPN intégré au pare-feu**

- Meilleur emplacement pour la fonction VPN est dans le pare-feu.

- **Avantages principaux**

- **Inspection complète du trafic** : analyse possible avant et après déchiffrement.
- **Conservation de toutes les fonctions de sécurité** :
 - Contrôle d'accès
 - Filtrage
 - Antivirus / antimalware, etc.
- **Centralisation de la sécurité** : les politiques de filtrage et de VPN sont gérées au même endroit.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Réseaux privés virtuels (VPN) - Alternatives moins efficaces

- **Sur périphérique interne (derrière le pare-feu)**
 - Trafic VPN chiffré traverse le pare-feu
 - Pare-feu incapable d'exécuter ses fonctions de sécurité
- **Sur routeur frontal (devant le pare-feu)**
 - Plateforme généralement moins sécurisée
 - Moins de fonctionnalités de sécurité disponibles

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Pare-feux distribués

Principe général:

Une configuration de pare-feu distribué implique des périphériques de pare-feu autonomes ou des pare-feux basés sur l'hôte travaillant ensemble sous un contrôle administratif central.

La figure 5 illustre cette configuration.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Pare-feux distribués

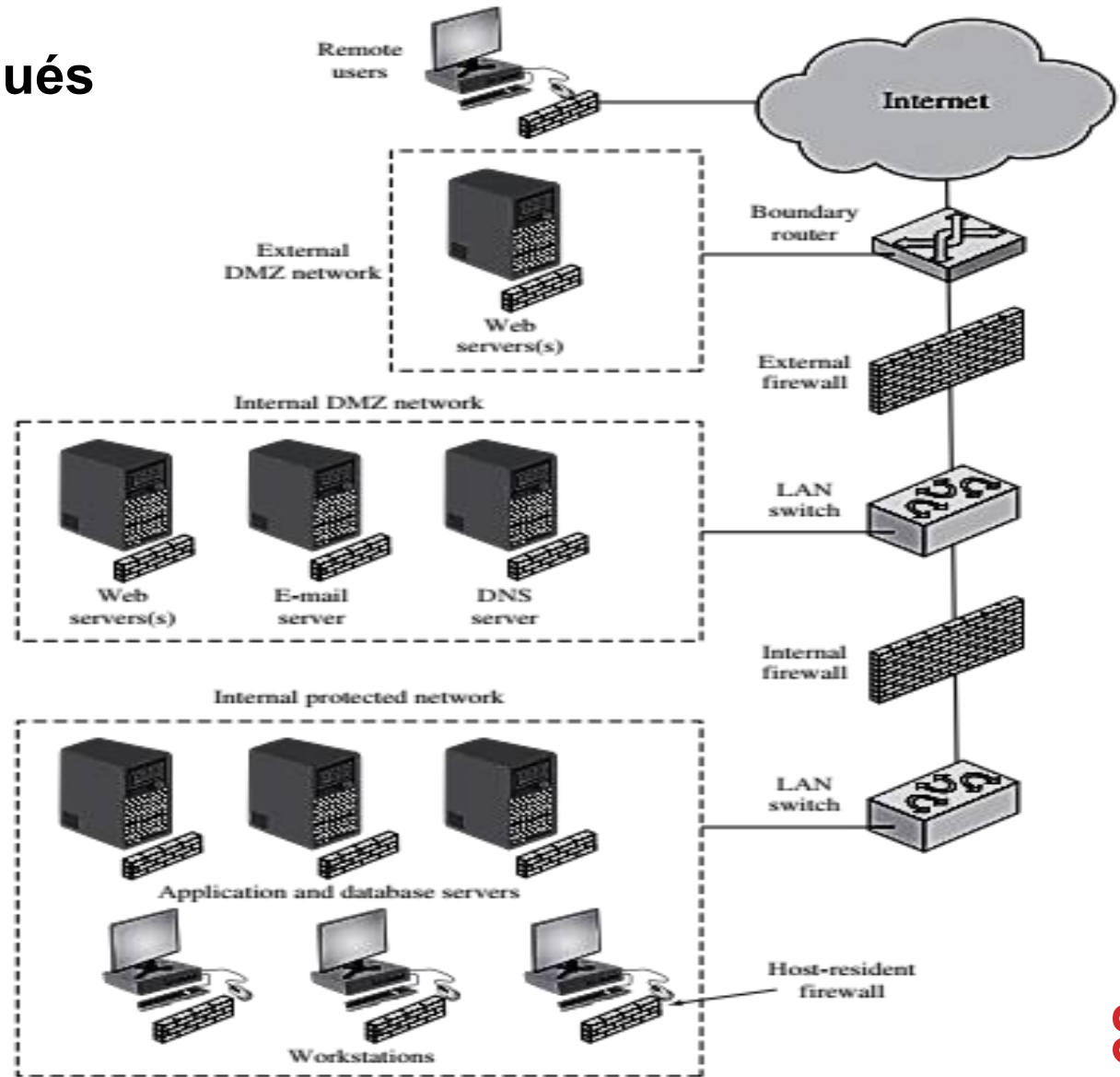


Fig. 5. Exemple de Pare-feu distribué [3]

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Pare-feux distribués

Avec des pare-feu distribués, on peut établir à la fois une DMZ interne et une DMZ externe.

- **DMZ externe** (devant le pare-feu externe)
 - Pour serveurs Web avec données moins critiques
 - Protection assurée par pare-feux basés sur l'hôte
- **DMZ interne** (entre pare-feux externe et interne)
 - Pour services nécessitant plus de protection
 - Bénéficie de la sécurité multicouche

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Pare-feux distribués

▪ Avantages de cette architecture:

- ✓ **Défense en profondeur** (plusieurs couches indépendantes).
- ✓ **Segmentation** selon le niveau de criticité des services.
- ✓ **Protection optimisée** : on adapte le niveau de sécurité aux besoins de chaque zone.

PARE-FEU : EMPLACEMENT ET CONFIGURATION

❑ Pare-feux distribués – Importance de la surveillance

Un **élément clé** dans une configuration de pare-feu distribué est la **surveillance de la sécurité**.

▪ Composantes essentielles:

- **Agrégation et analyse des journaux** : centralisation des logs et analyse des événements de sécurité
- **Collecte de statistiques** : suivi du trafic et mesure des événements.
- **Surveillance à distance ciblée** monitoring spécifique des hôtes critiques (serveurs sensibles, équipements clés,...).

▪ Bénéfices

- ✓ **Détection rapide** des anomalies.
- ✓ **Identification précoce** des menaces.
- ✓ **Vision globale et cohérente** de l'état de sécurité du réseau.

PARTIE 2

ANTIVIRUS ET ANTI-ANTIVIRUS

- Nous avons vu dans la première partie de ce cours que le pare-feu permet de faire face aux intrusions, mais il peut échouer de différentes manières.
- Dans ce cas, la ligne de défense suivante comprend les logiciels ou programmes antivirus, bien que certains d'entre eux luttent également contre les vers et les logiciels espions.

ANTIVIRUS ET ANTI-ANTIVIRUS

- Les virus essaient de se cacher et les utilisateurs essaient de les trouver,
→ un jeu du chat et de la souris !
- Dans ce qui suit, nous allons examiner quelques techniques de détection utilisées par les logiciels antivirus et aussi comment l'auteur du virus y répond.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ Détection à base de code (Signature)

Processus d'analyse :

Étape 1 : Isolation du virus

- Utilisation de **fichiers appâts** (*goat files*)
- Infection contrôlée de ces fichiers inoffensifs
- Obtention du virus dans sa forme la plus pure

Étape 2 : Création de signatures

- Établissement d'une liste exacte du code viral
 - Intégration dans la base de données des virus connus
 - Mise à jour des définitions antivirales
- ❖ Les entreprises se font concurrence sur la taille de leurs bases de données (Inventer de nouveaux virus .. !).

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Fonctionnement des programmes antivirus:

- **Analyse initiale**
 - Identification des menaces potentielles dès l'installation
 - Examen systématique de tous les fichiers exécutables
 - Comparaison avec la base de données de signatures virales
- **Mises à jour des définitions**
 - Téléchargement régulier depuis les sites des éditeurs
 - Intégration des signatures de virus nouvellement découverts
 - Protection continue contre les menaces émergentes

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Fonctionnement des programmes antivirus:

- **Optimisation des performances**
 - Algorithmes avancés pour accélérer l'analyse
 - Techniques d'indexation et de mise en cache des signatures
 - Solutions adaptées aux volumes importants (ex: 10 000 fichiers × 10 000 signatures)

Enjeu clé:

- ✓ Équilibre entre exhaustivité de l'analyse et rapidité d'exécution
- ✓ Minimisation de l'impact sur les performances système

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Deux types de recherche sont possibles :

- **Recherche exacte**

- correspondance exacte (**bit à bit**) avec le code du virus.
- Risque de faux négatifs (nouvelle variante de virus)

- **Recherche floue**

- Tolère des **variations** dans le code du virus.
- Réduit le risque qu'un virus modifié **échappe à la détection**.
- Plus lente et déclenche plus de fausses alarmes (faux positifs),

Donc, il faut faire un équilibre heuristique délicat.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

→ Stratégie des laboratoires

Identifier un code de base dans le virus qui est peu susceptible de changer et utilisez-le comme **signature virale** à rechercher.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

« Jeu du chat et de la souris »

Antivirus	Virus
Analyser fréquemment. L'analyse étant lente, il est plus efficace de vérifier uniquement les fichiers qui ont été modifiés depuis la dernière analyse.	
	Réinitialiser la date d'un fichier infecté à sa date d'origine pour éviter d'être détecté.
Vérifier la date de la dernière modification du répertoire du fichier.	
	Réinitialiser également la date du répertoire.
...	

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Autre façon de procéder :

Le programme antivirus enregistre et stocke sur le disque la longueur (taille) de tous les fichiers.

Si un fichier grandit depuis la dernière vérification, possible qu'il est infecté, comme illustré à la Fig. 6(a–b).

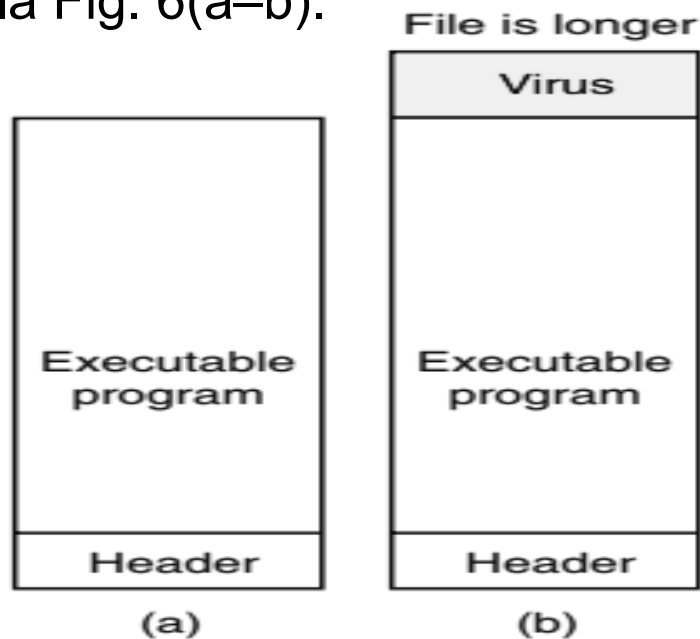


Fig. 6. Exemple de programme infecté [2]

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Cependant,

Un virus intelligent compresse le programme en le ramenant à sa taille initiale.

Pour que ce schéma fonctionne, le virus doit contenir à la fois des procédures de compression et de décompression.

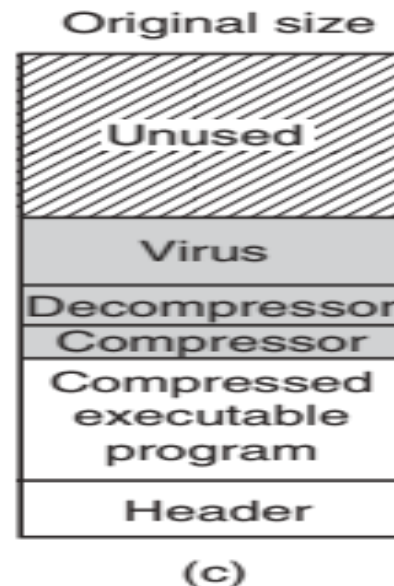


Fig. 7. Exemple de programme infecté compressé [2]

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Une autre façon:

Le virus modifie sa représentation sur le disque pour qu'elle ne corresponde pas à sa signature connue par l'antivirus.

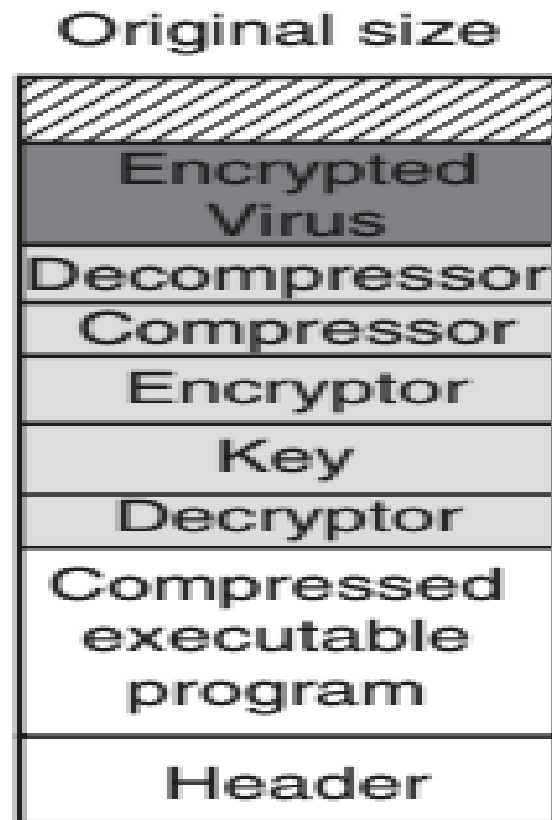
Technique utilisée:

Chiffrement du code avec une **clé différente** pour chaque fichier infecté.

- Génération d'une **clé de chiffrement aléatoire**(par exp. XOR entre l'heure actuelle et le contenu du fichier).
- **Chiffrement** du code du virus **mot par mot** via l'opération XOR.
- Stockage du **code chiffré** dans le fichier infecté.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code



(d)

Fig. 8. Virus crypté [2]

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Ce schéma reste imparfait car les fonctions de compression/décompression, et de cryptage/décryptage sont les mêmes dans toutes les copies.

- L'antivirus peut les utiliser comme signature de virus à rechercher.
- Donc, elles sont simplement cryptées avec le reste du virus, comme illustré à la Fig. 9 (e).

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

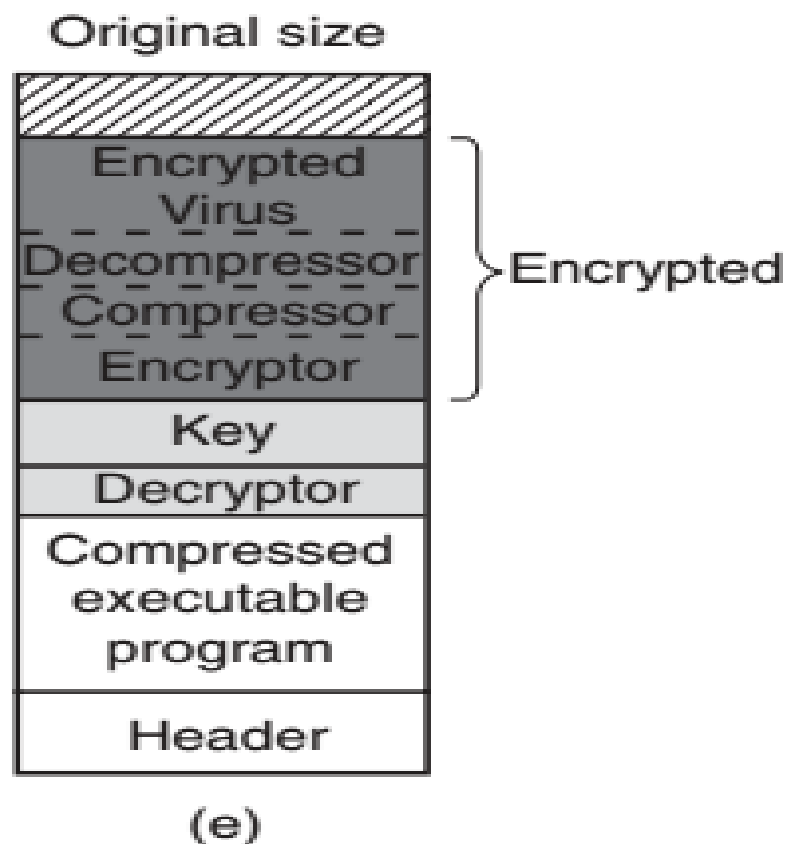


Fig. 9. Virus compressé avec code de compression crypté [2]

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Remarque :

- Le code de décryptage **ne peut pas être crypté**. Il doit s'exécuter en clair sur la machine pour décrypter le reste du virus.
 - Les antivirus ciblent la **fonction de décryptage**.
- Mais, ce n'est **pas la fin de l'histoire** !
 - D'autres techniques d'évasion existent...

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Par exemple, supposons que la fonction de décryptage doive effectuer le calcul suivant :

$$X = (A + B + C - 4)$$

Le code d'assemblage simple de ce calcul (pour un ordinateur générique à deux adresses) est le code (a) suivant :

```
MOV A,R1  
ADD B,R1  
ADD C,R1  
SUB #4,R1  
MOV R1,X
```

(a)

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

```
MOV A,R1  
NOP  
ADD B,R1  
NOP  
ADD C,R1  
NOP  
SUB #4,R1  
NOP  
MOV R1,X
```

(b)

Le code (b) fait la même chose, mais moins efficacement à cause des instructions NOP (*no operation*) intercalées avec le code réel.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

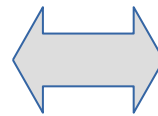
- Il est également possible de masquer le code de décryptage.
- Il existe de nombreuses façons de représenter NOP :
 - ajouter 0 à un registre,
 - l'associer avec lui-même par un OU,
 - le décaler de 0 bit à gauche et passer à l'instruction suivante.
- Sont toutes des façons de ne rien faire.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

```
MOV A,R1  
ADD B,R1  
ADD C,R1  
SUB #4,R1  
MOV R1,X
```

(a)



```
MOV A,R1  
ADD #0,R1  
ADD B,R1  
OR R1,R1  
ADD C,R1  
SHL #0,R1  
SUB #4,R1  
JMP .+1  
MOV R1,X
```

(c)

Par exemple, lors de la copie de lui-même, le virus peut utiliser le code (c) au lieu du code (a) et continuer à fonctionner plus tard lorsqu'il sera exécuté.

⇒ Virus polymorphe.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

```
MOV A,R1
ADD B,R1
ADD C,R1
SUB #4,R1
MOV R1,X
```

(a)

```
MOV A,R1
NOP
ADD B,R1
NOP
ADD C,R1
NOP
SUB #4,R1
NOP
MOV R1,X
```

(b)

```
MOV A,R1
ADD #0,R1
ADD B,R1
OR R1,R1
ADD C,R1
SHL #0,R1
SUB #4,R1
JMP .+1
MOV R1,X
```

(c)

```
MOV A,R1
OR R1,R1
ADD B,R1
MOV R1,R5
ADD C,R1
SHL R1,0
SUB #4,R1
ADD R5,R5
MOV R1,X
MOV R5,Y
```

(d)

```
MOV A,R1
TST R1
ADD C,R1
MOV R1,R5
ADD B,R1
CMP R2,R5
SUB #4,R1
JMP .+1
MOV R1,X
MOV R5,Y
```

(e)

Fig. 10. Exemple de virus polymorphe [2]

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

- **Problème:**

Pour un antivirus, il est **difficile** de détecter que différentes versions polymorphes d'un virus sont **équivalentes**.

- **Conséquences:**

- × Des **milliers de variantes** de virus à analyser,
- × **Temps de test insuffisant** ou **ralentissement important** du fonctionnement.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Détection à base de code

Remarque

- ❖ Un moteur de mutation performant (générant un bon **code polymorphe**) peut être un **véritable cauchemar** pour les antivirus.
- ❖ Le seul côté positif est que la **complexité de développement** de ces moteurs limite leur **prolifération** (pour l'instant ?).

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ A suivre...

- Une autre technique ??

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ Contrôle d'intégrité

- Le contrôle d'intégrité est une approche totalement différente de la détection des virus.
- Un programme antivirus qui fonctionne de cette manière analyse d'abord le disque dur à la recherche de virus. Une fois qu'il est convaincu que le disque est propre, il calcule une **somme de contrôle** pour chaque fichier exécutable.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Contrôle d'intégrité

Calcul des sommes de contrôle : Algorithmes selon le niveau de sécurité requis.

- Simple : addition de mots (entiers 32/64 bits)
- Avancé : hachage cryptographique non-inversible (SHA-256, etc.)

Processus de vérification

1. Première exécution

1. Calcul des sommes de contrôle pour tous les fichiers pertinents
2. Stockage dans un fichier de référence (checksum)

2. Exécutions suivantes

1. Recalcul des sommes de contrôle en temps réel
2. Comparaison avec les valeurs de référence stockées

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Contrôle d'intégrité

Le problème, c'est qu'il y a toujours une contre action de l'autre coté.

Stratégies d'attaque des virus:

- ✗ Suppression du fichier de contrôle.
- ✗ Falsification des sommes de contrôle.

Défis de protection:

- La simple dissimulation du fichier checksum est insuffisante.
- Les auteurs de virus peuvent analyser le fonctionnement des antivirus.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Contrôle d'intégrité

Solutions avancées:

- ✓ Stockage externe des sommes de contrôle (support non inscriptible)
- ✓ Signatures cryptographiques avec clés privées
- ✓ Mécanismes de détection des modifications du fichier checksum lui-même

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ Contrôle de comportement

- Une troisième stratégie utilisée par les logiciels antivirus est la vérification comportementale.
- Le programme antivirus analyse le **comportement** du fichier, pas son code.
- **Exemples de détection :**
 - Tentative d'écrasement du **secteur de démarrage**.
 - **Multiplication anormale** de connexions réseau.
 - **Modification des paramètres système** sans consentement.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Contrôle de comportement

▪ Principe

- Surveillance des actions des programmes en temps réel
- Détection basée sur les comportements suspects, non sur les signatures
- Capacité à identifier des menaces inconnues (zero-day)

▪ Forces : détection proactive

- Identification de nouveaux logiciels malveillants
- Protection contre les menaces émergentes
- Indépendance vis-à-vis des bases de signatures

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

Contrôle de comportement

- **Limitation majeure**
 - Classification erronée de programmes légitimes comme malveillants (faux positifs)
 - Perturbation potentielle des activités normales

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ Détection à base d'apprentissage automatique

▪ Évolution des antivirus

- Intégration des techniques de *Machine Learning* (ML).
- Dépassement des approches traditionnelles basées sur signatures.

▪ Méthodologie

- Exploitation d'algorithmes d'apprentissage sophistiqués.
- Traitement de vastes ensembles de données d'entraînement.
- Identification de patterns complexes invisibles aux méthodes classiques.

ANTIVIRUS ET TECHNIQUES ANTI-ANTIVIRUS

❑ Détection à base d'apprentissage automatique

▪ Avantages clés

- Détection proactive des menaces inconnues
- Adaptation continue aux nouvelles techniques malveillantes
- Réduction des faux positifs avec l'apprentissage progressif

▪ Prérequis

- Importante puissance de calcul disponible
- Ensembles de données d'entraînement de qualité
- Mise à jour régulière des modèles

PARTIE 3

SIGNATURE DE CODE

- Une autre stratégie complémentaire aux méthodes de détection de logiciels malveillants consiste à restreindre l'exécution aux logiciels dont l'**authenticité** et la **source** sont jugées fiables.
- Au lieu de chercher uniquement à **détecter et bloquer les malwares**, on **limite préventivement** ce qui a le droit de s'exécuter.

SIGNATURE DE CODE

Question clé :

Comment vérifier qu'un logiciel est **authentique** (provenant bien de l'éditeur légitime) et que son **intégrité** est préservée (aucune modification depuis la source) ?

- Enjeu majeur lors du téléchargement depuis des **sources partiellement fiables** :
 - magasins d'applications,
 - sites web à la **réputation inconnue**.
- Nécessité de mécanismes techniques :
 - **signatures numériques** pour l'authenticité,
 - **hachage / empreintes (SHA-256, etc.)** pour l'intégrité.

SIGNATURE DE CODE

- L'exécution de logiciels dont la **signature numérique est valide** et provient d'une **source de confiance** (éditeur/fournisseur légitime) réduit considérablement les risques liés aux logiciels malveillants.

SIGNATURE DE CODE

- La signature de code repose sur la **cryptographie à clé publique**.
- Processus de signature :
 - L'éditeur génère une paire de clés : clé publique disponible publiquement et clé privée gardée secrète.
 - Il calcule une valeur de hachage du code avec une fonction de hachage cryptographique (ex: SHA-256). La taille du hachage est standard (ex: 256 bits pour SHA-256).
 - Il signe ensuite la valeur de hachage avec sa clé privée.
 - La signature est jointe au logiciel.

SIGNATURE DE CODE

- Vérification côté récepteur :
 - L'utilisateur, en recevant le logiciel avec sa signature, applique la même fonction de hachage et enregistre la valeur obtenue.
 - L'utilisateur utilise la **clé publique de l'éditeur** pour effectuer l'opération de **vérification de la signature numérique** et **compare** ensuite la valeur du hash originale avec la valeur qu'il a obtenue.
 - Si les deux valeurs de hachage correspondent, le code est accepté comme authentique. Sinon, il est rejeté comme étant un faux.

Le processus de signature et de vérification est illustré à la figure 11.

SIGNATURE DE CODE

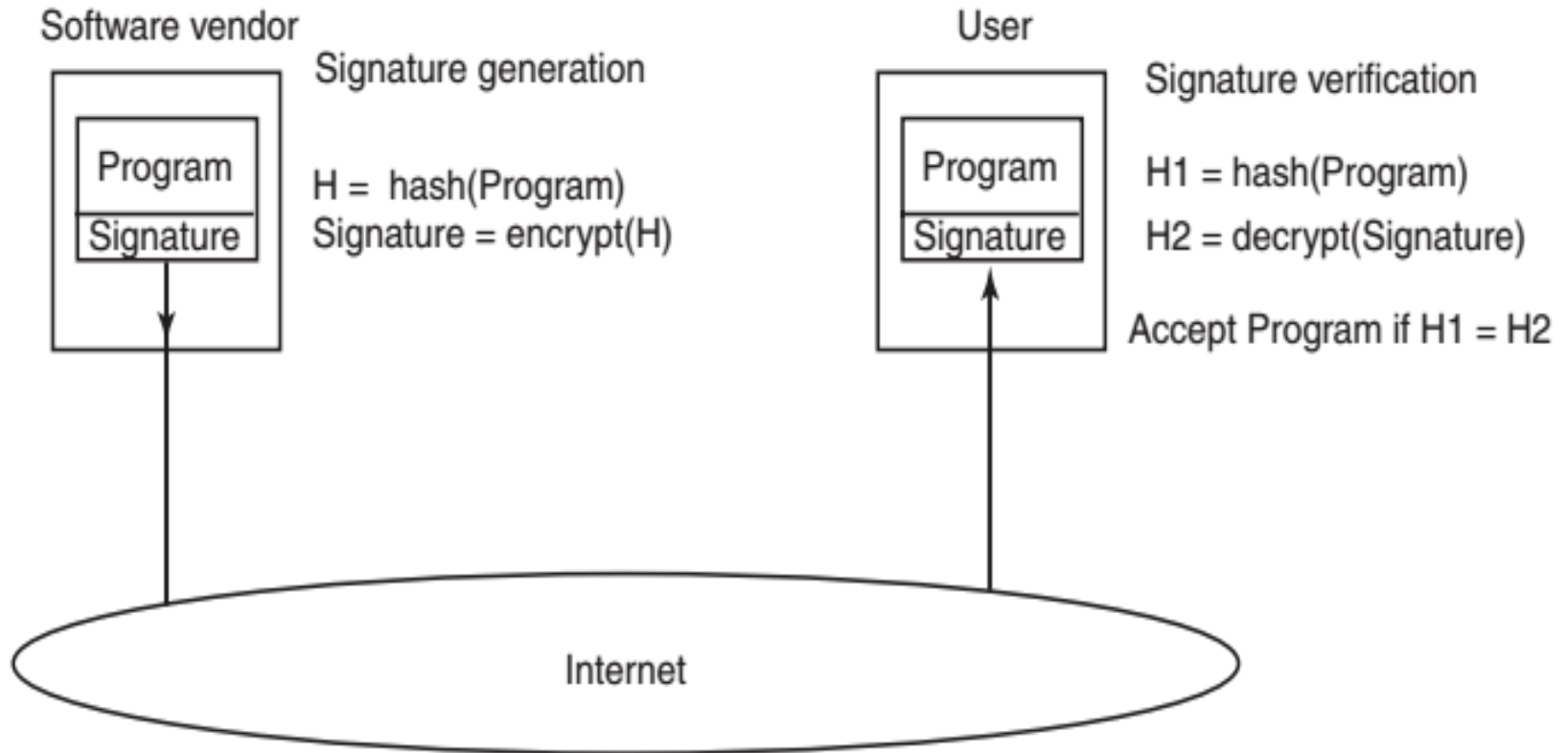


Fig. 11. Fonctionnement de la signature de code [2]

LE CONFINEMENT

- La signature de code prouve tout simplement d'où vient un programme, pas ce qu'il fait.
- Un logiciel signé par une source fiable peut néanmoins contenir des vulnérabilités ou être utilisé à des fins malveillantes.
- Des mécanismes de sécurité **complémentaires et multicouches** sont indispensables, tels que l'analyse comportementale qui a pour but d'évaluer et contrôler ce que fait le logiciel réellement.

Cette technique est appelée le confinement (*sandboxing*) tel qu'il est illustré à la figure 12.

LE CONFINEMENT

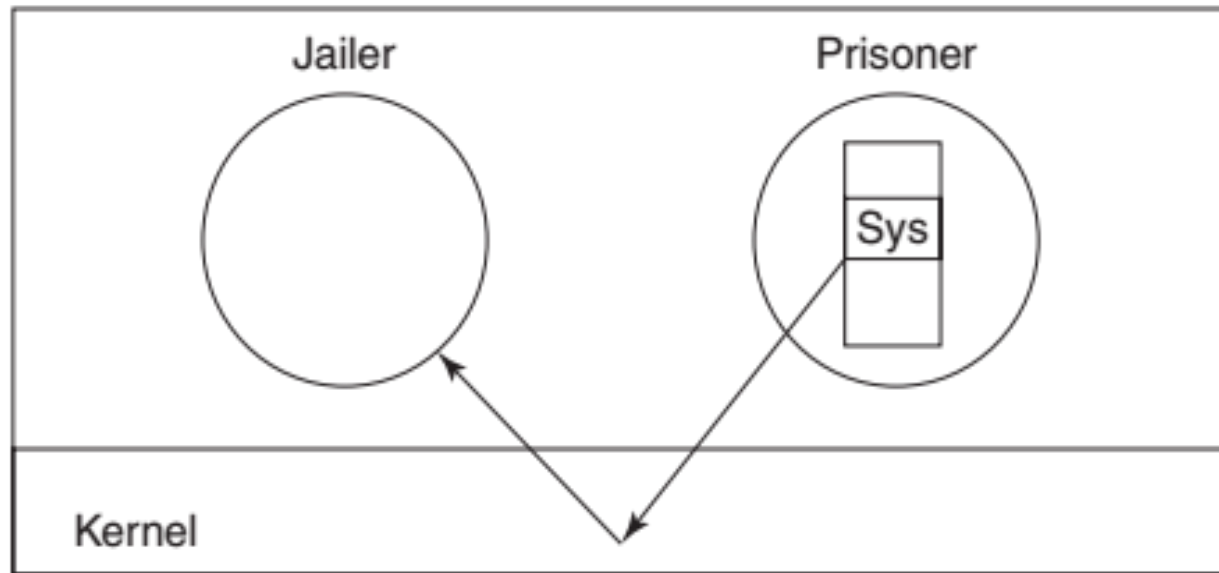
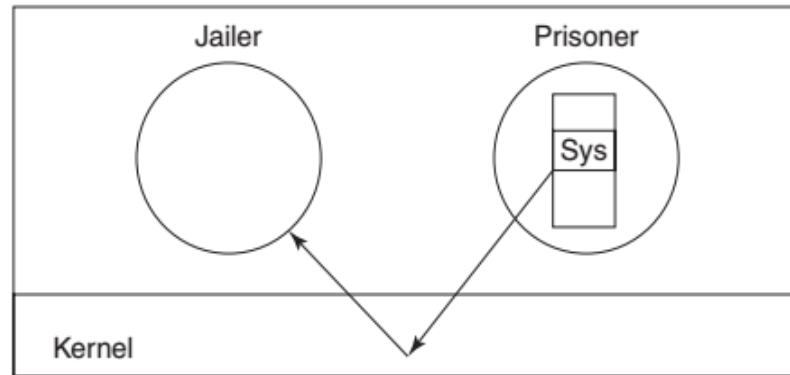


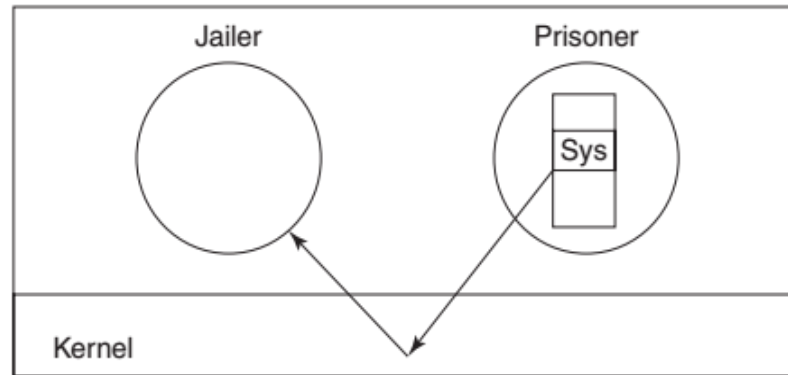
Fig. 12. Fonctionnement du confinement [2]

LE CONFINEMENT



- Un programme nouvellement acquis est exécuté en tant que **processus confiné**, qualifié de *prisonnier*.
- Ce processus est isolé du reste du système afin de limiter les risques en cas de comportement malveillant ou défaillant.
- Un second processus, de confiance et appartenant au système, agit comme un **geôlier** (*jailer*) chargé de surveiller le comportement du prisonnier.

LE CONFINEMENT



- Lorsqu'un processus confiné tente d'exécuter un **appel système**, celui-ci n'est pas immédiatement transmis au noyau.
- À la place, un mécanisme de déroutement intercepte l'appel et transfère le contrôle au geôlier. Ce dernier reçoit le numéro de l'appel système ainsi que ses paramètres, puis détermine, selon des règles de sécurité prédéfinies, si l'appel doit être autorisé, modifié ou bloqué.

LE CONFINEMENT

Par exemple :

- Si l'appel est **suspect** (ex. tentative de connexion réseau vers un hôte non autorisé) :
 - L'appel est **refusé**, et le processus peut être **tué**.
- Si l'appel est **autorisé** :
 - Le geôlier le **transmet au noyau** pour exécution.

De cette façon, un comportement erroné peut être détecté avant qu'il ne cause des problèmes.

LE CONFINEMENT

Avantages du confinement:

- Détection **précoce** des comportements suspects
- Empêche les programmes malveillants de nuire au système
- Renforce la **sécurité** des environnements d'exécution

Principe utilisé dans :

- ☐ Navigateurs Web
- ☐ Conteneurs (Docker)
- ☐ Applications mobiles

DÉTECTION D'INTRUSION

- Une autre stratégie de défense pour protéger une machine consiste à déployer un **système de détection d'intrusion** (*Intrusion Detection System*, IDS).
- Deux types principaux :
 - **HIDS** (*Host-based IDS*)
 - Surveille les activités internes d'un système (fichiers, processus, appels système).
 - **NIDS** (*Network-based IDS*)
 - Analyse le trafic réseau à la recherche de comportements suspects.

DÉTECTION D'INTRUSION

Dans ce qui suit,

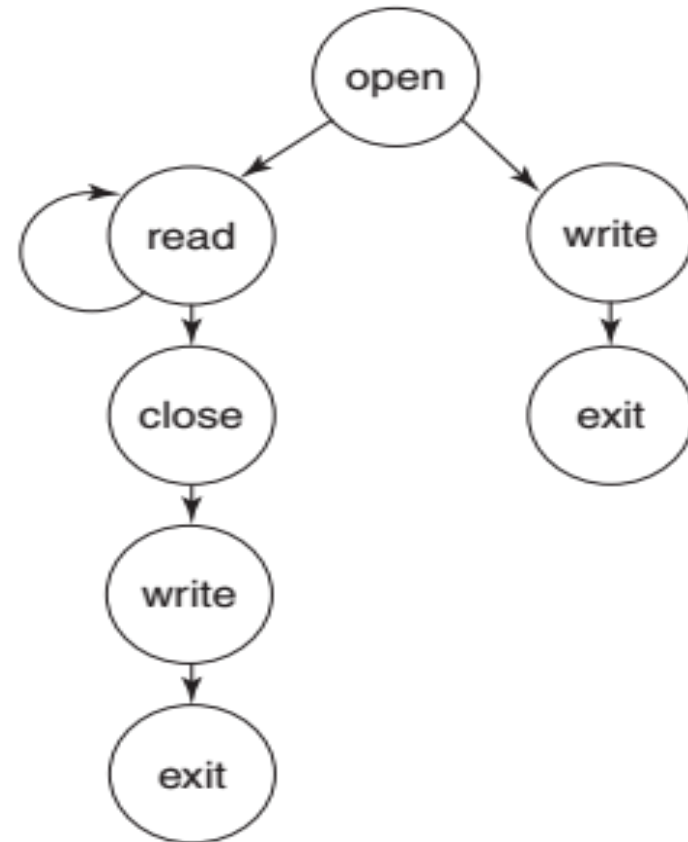
- Un type d'HIDS, appelé **détection d'intrusion statique basée sur un modèle**, sera décrit brièvement pour donner une idée de leur fonctionnement.
- Cette approche peut être mise en œuvre en utilisant la technique de confinement décrite précédemment.

DÉTECTION D'INTRUSION À BASE DE MODÈLE

```
int main(int argc *char argv[])
{
    int fd, n = 0;
    char buf[1];

    fd = open("data", 0);
    if (fd < 0) {
        printf("Bad data file\n");
        exit(1);
    } else {
        while (1) {
            read(fd, buf, 1);
            if (buf[0] == 0) {
                close(fd);
                printf("n = %d\n", n);
                exit(0);
            }
            n = n + 1;
        }
    }
}
```

(a)



(b)

Fig. 13. (a) Un programme, (b) Graphe des appels système de ce programme [2]

DÉTECTION D'INTRUSION À BASE DE MODÈLE

Comment peut-on interpréter ce graphe ?

- Le **premier appel système** effectué par le programme est toujours *open*, quelle que soient la condition d'exécution.
- Le **deuxième appel** dépend du succès de l'ouverture :
 - Si le fichier **n'a pas pu être ouvert**, l'appel suivant est *write* (pour afficher une erreur), suivi de *exit*.
 - Si le fichier **a été ouvert avec succès**, le second appel est *read*.
- En cas **de succès**, on peut observer :
 - Une **séquence de *read*** (lecture du contenu),
 - suivie d'un *close*, d'un *write* (affichage), puis d'un *exit*.

DÉTECTION D'INTRUSION À BASE DE MODÈLE

Donc,

- En fonctionnement normal (**sans attaque**), seule cette séquence est possible.
- Si le programme est **confiné** (sandboxing), le **geôlier** peut observer tous les appels système. Il lui est donc **facile de détecter une séquence anormale** (ex. : appel inattendu) et de **bloquer** le programme.

DÉTECTION D'INTRUSION À BASE DE MODÈLE

Hypothèse : Exécution d'un code malveillant

- Exploitation d'un **buffer overflow** dans un programme.
- Injection et exécution d'un **code non prévu**.

Comportement observé:

Le code malveillant génère une **séquence d'appels système différente** :

- Accès à des **fichiers non autorisés**.
- Tentative d'**ouverture de connexions réseau**.

Rôle du geôlier:

Surveille tous les appels système et **compare** chaque appel au modèle valide.

À la **première divergence**, il :

- **TUE** le processus.
- **ALERTE** l'administrateur système.

DÉTECTION D'INTRUSION À BASE DE MODÈLE

- Lorsque ce type de détection d'intrusion basée sur un **modèle statique** est utilisé, le geôlier doit connaître le modèle (c'est-à-dire le **graphe des appels système**).
- **Génération du modèle:**
 - ✓ Le **compilateur** peut produire ce modèle automatiquement.
 - ✓ L'auteur du programme **signe** le modèle avec son **certificat**.
- De cette manière, toute **modification ou falsification** du programme est détectée lors de son exécution, car le comportement réel ne correspondra pas au comportement attendu signé.
 - ✓ Le programme est **bloqué ou tué**.
 - ✓ Une **alerte** peut être déclenchée.

DÉTECTION D'INTRUSION À BASE DE MODÈLE

Cependant, il est possible de lancer ce qu'on appelle une **attaque par mimétisme** (*mimicry attack*), dans laquelle le code inséré effectue les mêmes appels système que le programme attaqué.

⇒ Dans ce cas, des **modèles plus sophistiqués** que ceux basés uniquement sur le simple suivi des appels système, sont nécessaires.

DÉTECTION D'INTRUSION - HONEYPOT

IDS par *Honeypot* : une autre approche de la détection

- Un IDS basé sur un **modèle comportemental** n'est pas le seul type d'IDS.
- De nombreux IDS reposent sur le concept de **pot de miel** (*honeypot*) :
 - Un leurre (appât) conçu pour **attirer et piéger** les pirates ou les logiciels malveillants.

DÉTECTION D'INTRUSION - HONEYPOT

Principe :

- Une machine **isolée** avec :
 - **Peu de défenses apparentes**
 - Un contenu **faussement précieux ou vulnérable**
- Dont l'objectif est:
 - **Observer discrètement** les tentatives d'intrusion.

A noter:

Certains IDS placent leurs pots de miel dans des machines virtuelles pour éviter d'endommager le système réel sous-jacent.

CONCLUSION

□ Enjeux de la Sécurité des Systèmes

- Les systèmes informatiques, dont le **SE** est la couche fondamentale, gèrent et stockent des actifs informationnels **critiques** et souvent **sensibles**.
- La sécurité de ces actifs repose sur la garantie de propriétés essentielles (la **triade CIA**) :
 - **Confidentialité**
 - **Intégrité**
 - **Disponibilité**
- Le SE doit assurer une sécurité robuste face à un paysage de menaces large et en constante évolution, incluant **divers types de malwares** :
 - **chevaux de Troie, virus, vers, logiciels espions et rootkits. ...**

CONCLUSION

❑ Stratégies et mécanismes de défense des systèmes

- Face aux menaces diverses et persistantes, les systèmes disposent d'un ensemble de **stratégies et de mécanismes** pour assurer leur protection.
- La stratégie fondamentale est la **défense en profondeur** (***Defense in Depth***) : une approche multicouche qui combine diverses techniques de sécurité pour créer une protection robuste.

CONCLUSION

❑ Stratégies et mécanismes de défense des systèmes

- Ces techniques incluent, par exemple :
 - **Protection au niveau du réseau/périmètre** : Pare-feux (filtrage, contrôle d'accès), Systèmes de détection/prévention d'intrusion réseau.
 - **Protection au niveau de l'hôte** : Antivirus et anti-malwares (NGAV), Pare-feux personnels.
 - **Sécurité logicielle** : Systèmes de signature de code (validation de l'authenticité/intégrité).
 - **Mécanismes d'isolement** : Sandboxing ou confinement (pour limiter l'impact de l'exécution de code potentiellement dangereux).
 - **Surveillance et analyse** : Systèmes de détection d'intrusion hôte (HIDS), Analyse de journaux (Logs), Pots de miel (Honeypots) pour la détection et la collecte d'informations sur les attaques.

**MERCI POUR VOTRE
ATTENTION.**

RÉFÉRENCES:

- [1] Andrew Tanenbaum « Systèmes d'exploitation » (Ed. 3). Pearson Education (2008).
- [2] ANDREW S. TANENBAUM & HERBERT BOS « MODERN OPERATING SYSTEMS » (4th Ed.) Pearson Education (2015).
- [3] William Stallings & Lawrie Brown, « Computer Security - Principles and Practice » Pearson Education Limited (2018).